

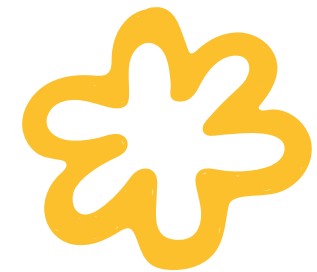


RUST

FUNCTION 😊



CONCEPT + SHORT CODE ILLUSTRATION

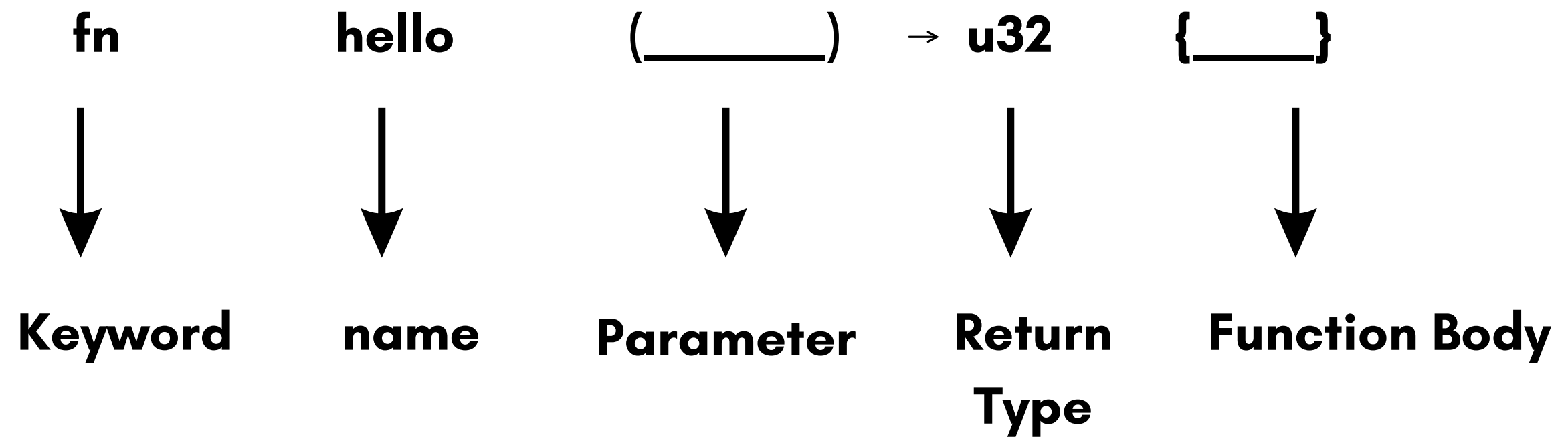


PART 1 - MEMBER 2

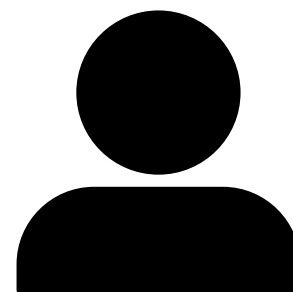
FUNCTION = ฟังก์ชันที่ทำงานบางอย่าง

Basic function

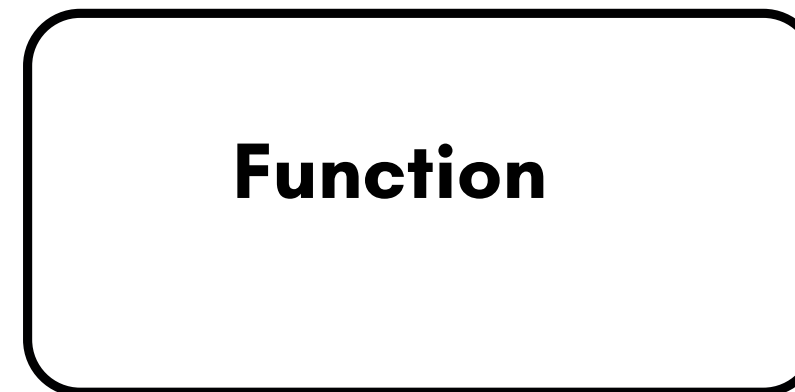
```
1 fn hello(){
2     println!("Hi bro");
3     println!("I'm nueng");
4     println!("Nice too meet u na ");
5 }
6 fn main(){
7     hello();
8     hello();
9     //-----
10    println!("Hi bro");
11    println!("I'm nueng");
12    println!("Nice too meet u na ");
13
14 }
```



input



Function



output



FUNCTION = ฟังก์ชันที่ทำงานบางอย่าง

Basic function

```
1  fn hello(){
2      println!("Hi bro");
3      println!("I'm nueng");
4      println!("Nice too meet u na ");
5  }
6  fn main(){
7      hello();
8      hello();
9      //-----
10     println!("Hi bro");
11     println!("I'm nueng");
12     println!("Nice too meet u na ");
13
14 }
```

output

```
Hi bro
I'm nueng
Nice too meet u na
Hi bro
I'm nueng
Nice too meet u na
Hi bro
I'm nueng
Nice too meet u na
```

Parameter , Argument ,Return

- **Parameter** = ตัวแปรที่ Function ใช้รับค่า
- **Argument** = ค่าจริงที่ส่งเข้า Function
- **Return** = ส่งค่ากลับ

```
fn greet(name: &str) {  
    println!("Hello {}", name);  
}
```

Parameter

```
fn add(a: i32, b: i32) {  
    println!("{}", a + b);  
}
```

Multiple Parameter

```
fn multiply(a: i32, b: i32) -> i32 {  
    a * b  
}
```

Multiple Parameter + return

► Run | ◉ Debug

```
fn main() {  
    greet(name: "John");  
    add(a: 10, b: 20);  
    let result: i32 = multiply(a: 5, b: 4);  
    println!("Result: {}", result);  
}
```

output

```
Hello John  
30  
Result: 20
```

ตัวอย่าง Data type

```
//let NetProfit: i32 = -20; i(n):n=8,16,32,64,128 (bit) คิดลบได้  
//let age: u32=7; U มีค่ามากกว่าเท่ากับ0  
//let price: f64 = 99.50;  
//let is_student: bool = true;  
//let grade: char = 'A';  
//let name: String = String::from("John");
```

Return = ส่งค่ากลับ



```
fn add(a: i32, b: i32) -> i32 {  
    a + b;  
}
```

ใส่ ; ทำให้ไม่ Return



```
fn add(a: i32, b: i32) -> i32 {  
    a + b  
}
```

or



```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}
```



```
fn check(x: i32) -> i32 {  
    if x > 10 {  
        100  
    } else {  
        "No"  
    }  
}
```

Return ผิด Type จากบางเงื่อนไข



```
fn test() -> i32 {  
    return 10;  
  
    println!("Hello");  
}
```

return แล้วมีโค้ดต่อ

```
fn main() {  
    let result = add(10, 20);  
  
    println!("{}", result);  
}
```

output

30

-> = บอกว่าจะ Return อะไร
return = สั่งให้ Returnทันที
บรรทัดสุดท้ายไม่มี ; =
Return อัตโนมัติ

But



```
fn count_down(n: i32) {  
    if n == 0 {  
        return;  
    }  
  
    println!("{}", n);  
    count_down(n - 1);  
}
```

```
fn main() {  
    count_down(3);  
}
```

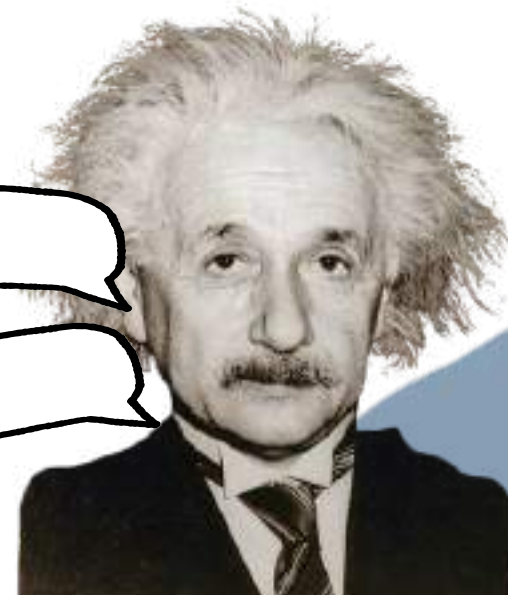
output

3
2
1

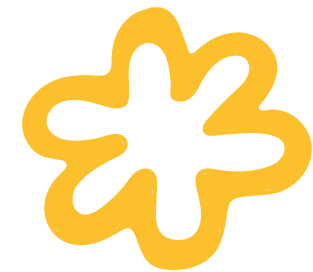
Recursive

So basic

What do you call this?



DETAILED CODE + LIVE DEMO



PART 2 - MEMBER 2

CODE OVERVIEW ADVANCED FUNCTION

struct

```
1 use std::io;
2 #[derive(Debug)]
3 struct Product {
4     name: String,
5     price: f64,
6 }
```

function

```
1 fn create_discount(discount: f64) -> impl Fn(f64) -> f64 {
2     move |price| price * (1.0 - discount)
3 }
4 fn calculate(
5     products: &[Product],
6     operation: impl Fn(f64) -> f64,
7 ) {
8     for product in products {
9         let new_price = operation(product.price);
10
11         println!(
12             "{} : {:.2} -> {:.2}",
13             product.name,
14             product.price,
15             new_price
16         );
17     }
18 }
19 fn input(message: &str) -> String {
20     let mut value = String::new();
21     println!("{}", message);
22     io::stdin().read_line(&mut value).unwrap();
23     value.trim().to_string()
24 }
25
```

main

```
1 fn main() {
2     let name1 = input("Product 1 name:");
3     let price1: f64 = input("Product 1 price:").parse().unwrap();
4
5     let name2 = input("Product 2 name:");
6     let price2: f64 = input("Product 2 price:").parse().unwrap();
7
8     let name3 = input("Product 3 name:");
9     let price3: f64 = input("Product 3 price:").parse().unwrap();
10
11     let discount: f64 = input("Discount (%):").parse().unwrap();
12
13     let products = [
14         Product {
15             name: name1,
16             price: price1,
17         },
18         Product {
19             name: name2,
20             price: price2,
21         },
22         Product {
23             name: name3,
24             price: price3,
25         },
26     ];
27
28     let discount_fn = create_discount(discount / 100.0);
29
30     calculate(&products, discount_fn);
31 }
```

ADVANCED FUNCTION

main

```
let discount_function: impl Fn(f64) -> f64 =  
    create_discount(discount / 100.0);
```

Function ที่ได้คืนมา "รับค่าเป็น f64"

Function ที่ได้คืนมา "คืนค่าเป็น f64"

create_discount() เป็น Fn ที่สร้าง Fn

```
fn create_discount(discount: f64) -> impl Fn(f64) -> f64 {  
    move |price: f64| price * (1.0 - discount)  
}
```

assume

Discount = 20 → create_discount(0.20)

จะสร้าง Closure ได้ move |price| price * (1.0 - 0.20)

ตอนนี้ discount_function ไม่ใช่ ราคาสินค้า ไม่ใช่ ตัวเลข
แต่เป็น Function

ประโยชน์ของการทำแบบนี้

discount10 → ลด 10%

discount20 → ลด 20%

discount50 → ลด 50%

แล้วเอาไปใช้กับราคาอะไรก็ได้

discount20(1000.0)

ได้ 800



ADVANCED FUNCTION

2. Function with Parameter + Fn

```
fn calculate(  
    products: &[Product],  
    operation: impl Fn(f64) -> f64,  
) {  
    for product: &Product in products {  
        let new_price: f64 = operation(product.price);  
  
        println!(  
            "{} : {:.2} -> {:.2}",  
            product.name,  
            product.price,  
            new_price  
        );  
    }  
}
```

main

```
calculate(&products, operation: discount_fn);
```

assume

product.price = 100 → operation(100) = discount_fn(100)
output 80

ADVANCED FUNCTION

3.Fn (input)

สร้าง string เปล่า

```
fn input(message: &str) -> String {  
    let mut value: String = String::new();  
    println!("{}", message);  
    io::stdin().read_line(buf: &mut value).unwrap();  
    value.trim().to_string()  
}
```

ถ้าเราเขียนแบบนี้ยาว เพราะ rust ไม่มีฟังก์ชันสั้น ๆ
รับค่าหน้าจอ เราเลยสร้างมันขึ้นมาเลย จบๆ

```
//let mut input = String::new();  
//io::stdin().read_line(&mut input).unwrap();  
//let age: i32 = input.trim().parse().unwrap();
```

Rust ไม่ได้ไม่ค่อยรับ input แต่ในการใช้งานจริง Rust มักถูกใช้ทำ
พวก backend ซึ่งไม่ได้หมายความว่าต้องรับค่าจากหน้าจอ
แบบโปรแกรมฝึกหัดเสมอไป

main

```
let name1: String = input(message: "Product 1 name:");  
let price1: f64 = input(message: "Product 1 price:").parse().unwrap();  
let name2: String = input(message: "Product 2 name:");  
let price2: f64 = input(message: "Product 2 price:").parse().unwrap();  
let name3: String = input(message: "Product 3 name:");  
let price3: f64 = input(message: "Product 3 price:").parse().unwrap();  
  
let discount: f64 = input(message: "Discount (%):").parse().unwrap();
```

input

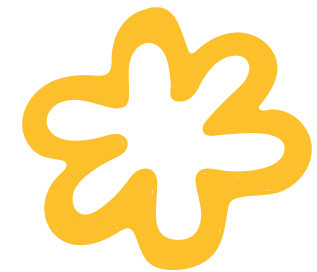
```
Product 1 name:  
ipad  
Product 1 price:  
9000  
Product 2 name:  
iphone  
Product 2 price:  
10000  
Product 3 name:  
macbook  
Product 3 price:  
90000  
Discount (%):  
60
```

output

```
ipad : 9000.00 -> 3600.00  
iphone : 10000.00 -> 4000.00  
macbook : 90000.00 -> 36000.00
```

RUST VS OTHER LANGUAGE

PPL ANALYSIS



PART 3 - MEMBER 3

ASPECT	RUST	PYTHON	JAVA	C++
SYNTAX	{ } กำหนด Block และมักใช้ ; ปิดท้าย Statement ประกาศ ตัวแปรด้วย let	Indentation กำหนด Block ไม่ ใช้ { } และไม่จำเป็นต้องใช้ ; ประกาศตัวแปรโดยกำหนดค่าได้ โดยตรง	{ } กำหนด Block และ ; ปิด ท้าย Statement ประกาศ ตัวแปรพร้อมชนิดข้อมูล	ใช้ { } กำหนด Block และ ; ปิดท้าย Statement ระบุชนิด ข้อมูลหรือใช้ auto
SEMANTIC	Compile เป็น Native Code และตรวจสอบ Type, Ownership, Borrowing ตั้งแต่ Compile Time	Dynamic Typing และข้อผิดพลาดบางประเภทจะตรวจพบ ขณะ Runtime	Compile เป็น Bytecode ทำงานผ่าน JVM และตรวจสอบ Type ตั้งแต่ Compile Time	Compile เป็น Native Code และให้ Programmer ควบคุม ทรัพยากรได้
TYPE SYSTEM	Statically Typed, Strongly Typed รองรับ Type Inference	Dynamically Typed, Strongly Typed	Statically Typed, Strongly Typed รองรับ Type Inference สำหรับ Local Variable	Statically Typed รองรับ Type Inference ผ่าน auto
MEMORY	ใช้ Ownership, Borrowing และ Lifetimes โดยไม่ใช้ Garbage Collector	ใช้ Automatic Memory Management ผ่าน Reference Counting ร่วมกับ Garbage Collector	ใช้ Garbage Collector (GC) จัดการ Memory ของ Object ที่ไม่ถูกใช้งาน	ใช้ Automatic Storage, RAII, Smart Pointers สามารถจัดการ Dynamic Memory โดยตรง
SAFETY	เน้น Memory Safety, Thread Safety โดยตรวจสอบตั้งแต่ Compile Time	โค้ดทั่วไปไม่ต้องจัดการ Raw Memory โดยตรง	JVM, GC ลดปัญหาการ จัดการ Memory โดยตรง	Raw Pointer และการจัดการ Memory ผิดพลาดอาจเกิด Memory Error

RUST VS PYTHON

RUST

- เน้น PERFORMANCE & MEMORY SAFETY
- ใช้ STATIC TYPING + OWNERSHIP
- ตรวจสอบ MEMORY ตั้งแต่ COMPILE TIME

PYTHON

- เน้น SIMPLICITY & FLEXIBILITY
- ใช้ DYNAMIC TYPING + AUTOMATIC MEMORY MANAGEMENT
- ข้อผิดพลาดบางอย่างพบตอน RUNTIME



KEY TAKEAWAY

RUST = PERFORMANCE + MEMORY SAFETY
โดยเน้นการตรวจสอบตั้งแต่ COMPILE TIME

ANALYSIS

RUST VS JAVA

ทั้งคู่เป็น STATICALLY TYPED แตกต่างหลักที่ MEMORY MANAGEMENT

RUST

- ใช้ OWNERSHIP & BORROWING จัดการ MEMORY
- ตรวจสอบ MEMORY ตั้งแต่ COMPILE RUST ไม่ใช้ GARBAGE COLLECTOR

JAVA

- ใช้ GARBAGE COLLECTOR (GC) จัดการ MEMORY อัตโนมัติขณะ RUNTIME

RUST VS C++

ทั้งคู่เน้น PERFORMANCE & RESOURCE CONTROL แตกต่างหลักที่ MEMORY SAFETY

RUST

- OWNERSHIP & BORROWING → COMPILER ตรวจสอบ

C++

- PROGRAMMER ควบคุม MEMORY/POINTER ได้มากกว่า
- ยืดหยุ่นกว่า แต่ต้องระวัง MEMORY ERRORS

PERSPECTIVE

SYNTAX

FUNCTION BASIC

```
fn function_name() {  
    // คำสั่งที่ต้องการให้ function ทำงาน  
}
```

FUNCTION ที่มี PARAMETER

```
fn function_name(parameter: Type) {  
    // function body  
}
```

FUNCTION ที่มี RETURN VALUE

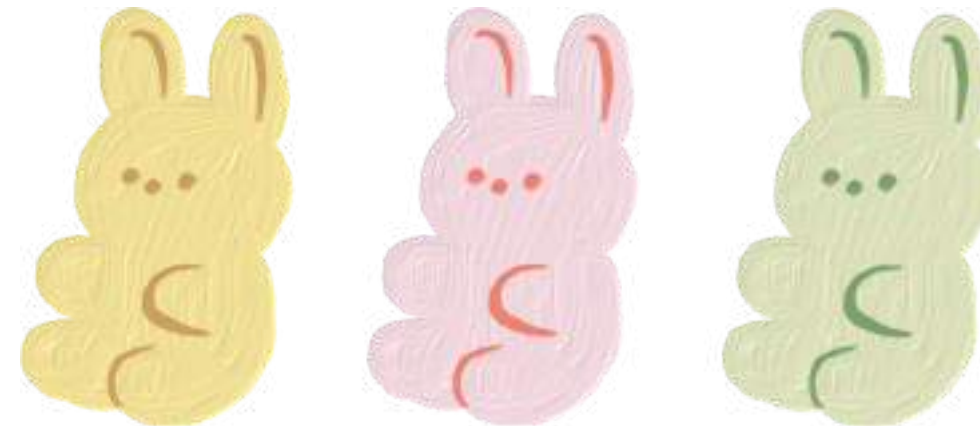
```
fn function_name() -> ReturnType {  
    value  
}
```

SEMANTIC

เมื่อมีการเรียก FUNCTION คำ ARGUMENTS จะถูกผูกกับ PARAMETERS

```
fn add_one(x: i32) -> i32 {  
    x + 1    // คำนวณค่า i32  
}
```

ถ้าใส่ SEMICOLON(;) จะทำให้ผลลัพธ์ของ EXPRESSION จะไม่ถูกใช้เป็นค่าที่ FUNCTION คืน ทำให้ BODY มีค่าเป็น () และเกิดเป็น ERROR



TYPE SYSTEM

STATICALLY TYPED

ชนิดข้อมูลของ PARAMETERS และ RETURN VALUE ของ FUNCTION จะถูกตรวจสอบตอน COMPILE ก่อนโปรแกรมทำงาน

♥ MEMORY / RESOURCE MANAGEMENT

ระบบ OWNERSHIP และ BORROWING

เมื่อส่งข้อมูลเข้า FUNCTION ค่าอาจถูก MOVE, COPY หรือ BORROW ขึ้นอยู่กับชนิดข้อมูลและวิธีการส่งค่า
เมื่อเจ้าของข้อมูลออกจาก SCOPE RUST จะทำลายข้อมูล และคืน RESOURCE โดยอัตโนมัติ

ABSTRACTION / OTHER PPL CONCEPTS



ABSTRACTION

PROCEDURAL ABSTRACTION ซึ่งผู้เรียกสนใจเพียงว่า FUNCTION รับอะไรเข้าไป และคืนอะไรออกมา โดยไม่จำเป็นต้องรู้รายละเอียดภายในทุกขั้นตอน

SCOPE

LEXICAL SCOPE / STATIC SCOPE พิจารณาได้จากโครงสร้างของ SOURCE CODE

BINDING

ค่า ARGUMENTS จะถูก BINDING เข้ากับ PARAMETERS ของ FUNCTION

PARADIGM

MULTI-PARADIGM รองรับทั้ง IMPERATIVE, FUNCTIONAL และ OBJECT-ORIENTED บางส่วน (ผ่าน STRUCT, IMPL, TRAIT)

OWNERSHIP & BORROWING

เมื่อส่งค่าเข้าไปใน FUNCTION ค่านั้นอาจถูก MOVE, COPY, BORROW ขึ้นอยู่กับชนิดข้อมูลและวิธีการส่งค่า โดยไม่จำเป็นต้องใช้ GARBAGE COLLECTOR

WHY RUST

SAFETY

ช่วยป้องกันข้อผิดพลาดด้านชนิดข้อมูลและปัญหาการจัดการหน่วยความจำ

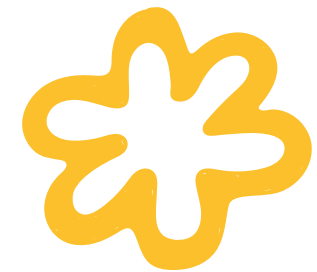
RELIABILITY

ช่วยให้พฤติกรรมของ FUNCTION คาดเดาได้มากขึ้น และตรวจพบข้อผิดพลาดหลายประเภทก่อนโปรแกรมทำงานจริง

PERFORMANCE

ส่งข้อมูลเข้า FUNCTION ผ่าน REFERENCE ช่วยให้ FUNCTION เข้าถึงข้อมูลได้โดยไม่จำเป็นต้องคัดลอกข้อมูลทั้งหมด และจัดการหน่วยความจำได้โดยไม่ต้องพึ่ง GARBAGE COLLECTOR

EXERCISES + COMMON MISTAKES



PART 4 - MEMBER 4

COMMON MISTAKES

```
fn add(x:i32 , y:i32) -> i32{
    x + y
}

fn add(x: f64, y: f64) -> f64 {
    x + y
}

fn main() {
    let sum1 = add(2,5);
    let sum2 = add(2.2,5.5);
    println!("sum1 = {}",sum1);
    println!("sum2 = {}",sum2);
}
```

COMMON MISTAKES

✗ Incorrect

```
fn add(x:i32 , y:i32) -> i32{
    x + y
}

fn add(x: f64, y: f64) -> f64 {
    x + y
}

fn main() {
    let sum1 = add(2,5);
    let sum2 = add(2.2,5.5);
    println!("sum1 = {}",sum1);
    println!("sum2 = {}",sum2);
}
```

✓ Correct

```
fn add_i32(x:i32 , y:i32) -> i32{
    x + y
}

fn add_f64(x: f64, y: f64) -> f64 {
    x + y
}

fn main() {
    let sum1 = add_i32(2,5);
    let sum2 = add_f64(2.2,5.5);
    println!("sum1 = {}",sum1); // sum1 = 7
    println!("sum2 = {}",sum2); // sum2 = 7.7
}
```

ทำไมถึงผิด? (Why?)

ภาษา rust **ไม่รองรับการทำ Function Overloading** (การสร้างฟังก์ชันชื่อเดียวกัน แต่รับ Parameter หรือมี Type ต่างกัน)

- เพื่อป้องกันความสับสนของ Type Inference : หากมี Overloading อาจทำให้ Compiler สับสนหรือคาดเดาประเภทข้อมูลผิดจากที่ผู้เขียนตั้งใจ

COMMON MISTAKES

```
fn greet(name: &str, msg: &str = "Hello") {  
    println!("{}", {}, msg, name);  
}  
fn main() {  
    greet("Alice", None);  
    greet("Bob", Some("Good morning"));  
}
```

COMMON MISTAKES

✗ Incorrect

```
fn greet(name: &str, msg: &str = "Hello") {  
    println!("{}", {}, msg, name);  
}  
fn main() {  
    greet("Alice", None);  
    greet("Bob", Some("Good morning"));  
}
```

✓ Correct

```
fn greet(name: &str, msg: Option<&str>) {  
    let greeting = match msg {  
        Some(msg) => msg,  
        None => "Hello",  
    };  
    println!("{}", {}, greeting, name);  
}  
fn main() {  
    greet("Alice", None); // Hello, Alice!  
    greet("Bob", Some("Good morning")); // Good morning, Bob!  
}
```

ทำไมถึงผิด? (Why?)

ภาษา rust ไม่รองรับการทำ default parameter (การกำหนดค่าเริ่มต้นให้กับฟังก์ชัน)

- เพื่อความชัดเจน (Explicit over Implicit): Rust เน้นให้ผู้อ่านได้มองเห็นสิ่งที่เกิดขึ้นอย่างชัดเจนที่สุด โดยไม่มีระบบซ่อนการทำงานอยู่เบื้องหลัง

CHALLENGE 1

จงเขียนฟังก์ชันชื่อ `is_even` ที่รับตัวเลขจำนวนเต็ม (i32) เข้ามา 1 ตัว แล้วทำการตรวจสอบว่าตัวเลขนั้นเป็น เลขคู่ หรือไม่

- หากเป็นเลขคู่ ให้พิมพ์คำว่า "True"
- หากเป็นเลขคี่ ให้พิมพ์คำว่า "False"

// ไม่มีการ return (ไม่ต้องใส่ -> ...)



HINT

- สามารถตรวจสอบเลขคู่ โดยใช้ตัวดำเนินการ Modulo (%)



CHALLENGE 1

จงเขียนฟังก์ชันชื่อ `is_even` ที่รับตัวเลข
จำนวนเต็ม (`i32`) เข้ามา 1 ตัว แล้วทำการ
ตรวจสอบว่าตัวเลขนั้นเป็น เลขคู่ หรือไม่

- หากเป็นเลขคู่ ให้พิมพ์คำว่า `"True"`
- หากเป็นเลขคี่ ให้พิมพ์คำว่า `"False"`

// ไม่มีการ return (ไม่ต้องใส่ -> ...)

✓ Solution

```
fn is_even(number: i32){  
    if number % 2 == 0 {  
        println!("True");  
    } else {  
        println!("False");  
    }  
}  
  
fn main() {  
    let num1 = 4;  
    let num2 = 7;  
  
    is_even(num1); // True  
    is_even(num2); // False  
}
```



CHALLENGE 2

จงเขียนฟังก์ชันชื่อ `grade_check` โดยรับพารามิเตอร์เป็นตัวเลขจำนวนเต็ม `score (i32)` จากนั้นจะตัดเกรดโดยใช้เกณฑ์ว่า ถ้าได้ไม่ถึง 40 คะแนนได้เกรดเป็น `Fail`, ได้ถึง 40 แต่ไม่ถึง 80 คะแนนได้เกรดเป็น `Pass`, และถ้าได้ ถึง 80 คะแนน จะได้เกรดเป็น `Excellent`

// คะแนนที่รับมาจะไม่เป็นค่าติดลบ หรือมากกว่า 100

// ไม่มีการ return (ไม่ต้องใส่ -> ...)



CHALLENGE 2

จงเขียนฟังก์ชันชื่อ `grade_check` โดยรับพารามิเตอร์เป็นตัวเลขจำนวนเต็ม `score (i32)` จากนั้นจะตัดเกรดโดยใช้เกณฑ์ว่า ถ้าได้ไม่ถึง 40 คะแนนจะได้เกรดเป็น **Fail**, ได้ถึง 40 แต่ไม่ถึง 80 คะแนนจะได้เกรดเป็น **Pass**, และถ้าได้ ถึง 80 คะแนน จะได้เกรดเป็น **Excellent**

// คะแนนที่รับมาจะไม่เป็นค่าติดลบ หรือมากกว่า 100
// ไม่มีการ return (ไม่ต้องใส่ -> ...)

✓ Solution

```
fn grade_check(score: i32){  
    if score >= 80 {  
        println!("Excellent");  
    } else if score >= 40 {  
        println!("Pass");  
    } else {  
        println!("Fail");  
    }  
}  
  
fn main() {  
    grade_check(39); // Fail  
    grade_check(40); // Pass  
    grade_check(90); // Excellent  
}
```

KEY TAKEAWAYS – สรุปสิ่งที่ควรจำ

1

ประกาศ function ด้วย `fn name(parameter: datatype) { ... }`

2

ต้องระบุ `type` ของ `parameter` ทุกตัวและ ถ้ามีการคืนค่า ให้ `return type` ด้วย `→ ...`

3

บรรทัดสุดท้ายที่ไม่มี `;` คือค่าที่ `return`

4

ภาษา rust ไม่รองรับการทำ `Function Overloading` และ `default parameter`



REFERENCES – เอกสารอ้างอิง

- https://www.w3schools.com/rust/rust_functions.php
- <https://www.geeksforgeeks.org/blogs/rust-vs-python/>
- <https://youtu.be/g2qJh-7AiW4?si=fOgnEv-wkoUpePpO>
- Example ideas---
https://webserv.cp.su.ac.th/lecturer/pinyotae/compro1/labs/lab_problem_set.pdf



MEMBER

นายกฤตเมธ ไทยภักดี	670710121	Concept + Short Code + Detailed Code + Live Demo
นางสาวกัญญารัตน์ สมหวังพรเจริญ	670710122	Rust vs Other Language + PPL Analysis
นางสาวกัญญาณัฐ เขมนรากร	670710123	Exercises + Common Mistakes

GROUP 4

QUESTIONS

&

ANSWER